

Tareas de Git

-----|GIT 1|-----

- Instalar Git:

```
osboxes@osboxes:~$ sudo apt-get install git
[sudo] password for osboxes:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  libdbus-glib-1-2
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
  git-man liberror-perl patch
Suggested packages:
```

- Crear carpeta, y configurar las variables globales.

```
osboxes@osboxes:~$ ls
Desktop    Downloads  Pictures   Templates  webmin-setup-repos.sh
Documents  Music      Public     Videos
osboxes@osboxes:~$ sudo mkdir practicas_Git
osboxes@osboxes:~$ ls
Desktop    Downloads  Pictures   Public     Videos
Documents  Music      practicas_Git  Templates  webmin-setup-repos.sh
osboxes@osboxes:~$ git config --global user.name Judit
osboxes@osboxes:~$ git config --global user.email jquivio1606@g.educaand.es
osboxes@osboxes:~$ cd practicas_Git/
```

- Hacer un git init, es decir, crear un repositorio local.

```
osboxes@osboxes:~/practicas_Git$ sudo git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/osboxes/practicas_Git/.git/
```

- Cambiarle el nombre a la rama principal, que al hacer git init se llama 'master', pero cuando se clona un repositorio de gitHub, se llama 'main'.

```
osboxes@osboxes:~/practicass_Git$ sudo git branch -m main
```

- Hacer un git status, ver que no hay nada que subir.

```
osboxes@osboxes:~/practicass_Git$ sudo git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

- Crear un archivo y ver que el git status te dice que hay un archivo creado pero no está subido al repositorio local.

```
osboxes@osboxes:~/practicass_Git$ echo "Archivo 1 para hacer el git add, y git commit" >> archivo1.txt
```

```
osboxes@osboxes:~/practicass_Git$ sudo git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    archivo1.txt

nothing added to commit but untracked files present (use "git add" to track)
```

- Hacer un git add, y un commit para subir el archivo recién creado, y ver que ya no hay nada para hacer commit.

```
osboxes@osboxes:~/practicass_Git$ sudo git add .
```

```
osboxes@osboxes:~/practicass_Git$ sudo git commit -m "Primer Commit"
[main (root-commit) 9297e0c] Primer Commit
 1 file changed, 1 insertion(+)
 create mode 100644 archivo1.txt
```

```
osboxes@osboxes:~/practicass_Git$ git status
On branch main
nothing to commit, working tree clean
```

- Crear otro archivo para comprobar el git log.

```
osboxes@osboxes:~/practicass_Git$ sudo echo "Archivo 2 para hacer el git log, y git status" >> archivo2.txt
```

- Como todavía no he hecho commit del segundo archivo, solo hay un log.

```
osboxes@osboxes:~/practicass_Git$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        archivo2.txt

nothing added to commit but untracked files present (use "git add" to track)
osboxes@osboxes:~/practicass_Git$ git add .
```

```
osboxes@osboxes:~/practicass_Git$ sudo git log
commit 9297e0c9ba076a2d65d9b3c66fed2f93ee39ce94 (HEAD -> main)
Author: Judit <jquivio1606@g.educaand.es>
Date:   Tue Feb 25 13:15:22 2025 -0500

    Primer Commit
```

- Si hacemos un git add, y un git commit, y volvemos a ejecutar los comandos: git status, y git log, veremos que ahora tenemos dos logs.

```
osboxes@osboxes:~/practicass_Git$ sudo git add .
```

```
osboxes@osboxes:~/practicass_Git$ sudo git commit -m "Subir archivo2.txt - Segundo commit"
[main 5a42536] Subir archivo2.txt - Segundo commit
1 file changed, 1 insertion(+)
create mode 100644 archivo2.txt
```

```
osboxes@osboxes:~/practicass_Git$ git status
On branch main
nothing to commit, working tree clean
osboxes@osboxes:~/practicass_Git$ sudo git log
commit 5a425366209cf41560ed791f88d653233a99e7b4 (HEAD -> main)
Author: Judit <jquivio1606@g.educaand.es>
Date:   Tue Feb 25 13:19:22 2025 -0500

    Subir archivo2.txt - Segundo commit

commit 9297e0c9ba076a2d65d9b3c66fed2f93ee39ce94
Author: Judit <jquivio1606@g.educaand.es>
Date:   Tue Feb 25 13:15:22 2025 -0500

    Primer Commit
```

- Para cambiar de versión (de commit) ponemos el comando git checkout "Identificador del commit". En este caso me muevo al primer commit.

```
osboxes@osboxes:~/practicass_Git$ sudo git checkout 9297e0c9ba076a2d65d9b3c66fed2f93ee39ce94
Note: switching to '9297e0c9ba076a2d65d9b3c66fed2f93ee39ce94'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 9297e0c Primer Commit
osboxes@osboxes:~/practicass_Git$ git log --oneline
9297e0c (HEAD) Primer Commit
osboxes@osboxes:~/practicass_Git$ git status
HEAD detached at 9297e0c
nothing to commit, working tree clean
```

- Para volver al commit normal, ponemos el comando git reset y al id del último commit. (En verdad creo que no ha funcionado pero es que poniendo solo git reset solo no pasa nada).

```
osboxes@osboxes:~/practicass_Git$ sudo git reset c597453
Unstaged changes after reset:
D      archivo2.txt
D      archivo3.txt
osboxes@osboxes:~/practicass_Git$ git log --oneline
c597453 (HEAD, main) Subir archivo3.txt - Tercer commit
5a42536 Subir archivo2.txt - Segundo commit
9297e0c Primer Commit
osboxes@osboxes:~/practicass_Git$ sudo git log --graph
* commit c597453768857c4c2e3044680a0b575e35c4d755 (HEAD, main)
| Author: Judit <jquivio1606@g.educaand.es>
| Date:   Tue Feb 25 13:33:46 2025 -0500
|
|       Subir archivo3.txt - Tercer commit
|
* commit 5a425366209cf41560ed791f88d653233a99e7b4
| Author: Judit <jquivio1606@g.educaand.es>
| Date:   Tue Feb 25 13:19:22 2025 -0500
|
|       Subir archivo2.txt - Segundo commit
|
* commit 9297e0c9ba076a2d65d9b3c66fed2f93ee39ce94
| Author: Judit <jquivio1606@g.educaand.es>
| Date:   Tue Feb 25 13:15:22 2025 -0500
|
|       Primer Commit
```

- Para poner un alias a los commit tenemos que poner los comandos de la captura.

```
osboxes@osboxes:~/practicass_Git$ sudo git config --global alias.tree "log --graph --decorate --all --oneline"
osboxes@osboxes:~/practicass_Git$ sudo git tree
* c597453 (HEAD, main) Subir archivo3.txt - Tercer commit
* 5a42536 Subir archivo2.txt - Segundo commit
* 9297e0c Primer Commit
```

- Para probar el gitignore creamos un archivo que es el que queremos que no se suba.

```
osboxes@osboxes:~/practicass_Git$ echo "Archivo4.txt para probar el .gitignore" >> archivo4.txt
```

- Si hacemos un git status vemos las cosas que no están subidas.

```
osboxes@osboxes:~/practicass_Git$ git status
HEAD detached from 9297e0c
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    archivo2.txt
        deleted:    archivo3.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        archivo4.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

- Creamos el archivo .gitignore, y ponemos el archivo que no queremos subir en el archivo. Si ahora hacemos un add, commit, y un git status vemos como ya no hay nada que subir.

```
osboxes@osboxes: ~/practicass_Git
GNU nano 7.2                                .gitignore *
archivo4.txt
```

```
osboxes@osboxes:~/practicass_Git$ sudo git add .
osboxes@osboxes:~/practicass_Git$ sudo git commit -am "Subir el gitignore"
[detached HEAD c11139e] Subir el gitignore
 3 files changed, 1 insertion(+), 2 deletions(-)
 create mode 100644 .gitignore
 delete mode 100644 archivo2.txt
 delete mode 100644 archivo3.txt
osboxes@osboxes:~/practicass_Git$ sudo git status
HEAD detached from 9297e0c
nothing to commit, working tree clean
```

- Como último comando de la tarea Git 1: Modificamos el archivo1.txt añadiendo otra línea.

```
osboxes@osboxes:~/practicas_Git$ sudo nano archivo1.txt
```

```
osboxes@osboxes: ~/practicas_Git
GNU nano 7.2                                archivo1.txt *
Archivo 1 para hacer add y commit
Ahora vamos a probar el git diff
```

- Si ponemos un git status vemos como está modificado y si hacemos un git diff, señala la diferencia entre el archivo que está subido, y el modificado de ahora.

```
osboxes@osboxes:~/practicas_Git$ git status
HEAD detached from 9297e0c
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   archivo1.txt

no changes added to commit (use "git add" and/or "git commit -a")
osboxes@osboxes:~/practicas_Git$ sudo git diff archivo1.txt
diff --git a/archivo1.txt b/archivo1.txt
index f06c0c5..11b201b 100644
--- a/archivo1.txt
+++ b/archivo1.txt
@@ -1,2 @@
  Archivo 1 para hacer add y commit
+Ahora vamos a probar el git diff
```

-----|GIT 2|-----

(Lo de Git 1 lo hice en clase, y en debian, la parte de git 2 y git 3 las hice en mi casa en windows, con el git bash).

Hacemos un checkout al hash del primer commit:

Con esto vemos como se han borrado los commits anteriores, y los archivos.

```
MINGW64:/d/DAW/Repositorio/2DAW/practicas_Git
Judith@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git log --oneline
b437209 (HEAD -> main) Cuarto commit: Ignorar archivo4.txt - .gitignore
d7e0985 Tercer commit - archivo3.txt
5f2b6ae Segundo commit - archivo2.txt
3b3463c Primer Commit

Judith@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git checkout 3b3463c
Note: switching to '3b3463c'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 3b3463c Primer Commit

Judith@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git ((3b3463c...))
$ git log --oneline
3b3463c (HEAD) Primer Commit
```



```
MINGW64:/d/DAW/Repositorio/2DAW/practicas_Git
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git ((3b3463c...))
$ git branch
* (HEAD detached at 3b3463c)
  main

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git ((3b3463c...))
$ ls
archivo1.txt  archivo4.txt
```

Si ahora volvemos a poner el hash del último commit, volvemos a tener los archivos que teníamos antes.

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git ((3b3463c...))
$ git checkout b437209
Previous HEAD position was 3b3463c Primer Commit
HEAD is now at b437209 Cuarto commit: Ignorar archivo4.txt - .gitignore

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git ((b437209...))
$ git log --oneline
b437209 (HEAD, main) Cuarto commit: Ignorar archivo4.txt - .gitignore
d7e0985 Tercer commit - archivo3.txt
5f2b6ae Segundo commit - archivo2.txt
3b3463c Primer Commit

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git ((b437209...))
$ git branch
* (HEAD detached at b437209)
  main

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git ((b437209...))
$ ls
archivo1.txt  archivo2.txt  archivo3.txt  archivo4.txt
```

Si queremos volver a un commit determinado también podemos poner esto:

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git ((b437209...))
$ git reset --hard d7e0985
HEAD is now at d7e0985 Tercer commit - archivo3.txt

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git ((d7e0985...))
$ git log --oneline
d7e0985 (HEAD) Tercer commit - archivo3.txt
5f2b6ae Segundo commit - archivo2.txt
3b3463c Primer Commit
```

Con el git reflog, vemos todos los movimientos que hemos hecho:

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git ((d7e0985...))
$ git reflog
d7e0985 (HEAD) HEAD@{0}: reset: moving to d7e0985
b437209 (main) HEAD@{1}: checkout: moving from 3b3463c92a4533a89b52598954e2db34f
a280b43 to b437209
3b3463c HEAD@{2}: checkout: moving from main to 3b3463c
b437209 (main) HEAD@{3}: commit: Cuarto commit: Ignorar archivo4.txt - .gitignore
e
d7e0985 (HEAD) HEAD@{4}: commit: Tercer commit - archivo3.txt
5f2b6ae HEAD@{5}: commit: Segundo commit - archivo2.txt
3b3463c HEAD@{6}: commit (initial): Primer Commit
```

Si queremos volver al punto donde lo dejamos podemos hacer un git checkout main y ya estaría.

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git ((d7e0985...))
$ git checkout main
Previous HEAD position was d7e0985 Tercer commit - archivo3.txt
Switched to branch 'main'

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git log --oneline
b437209 (HEAD -> main) Cuarto commit: Ignorar archivo4.txt - .gitignore
d7e0985 Tercer commit - archivo3.txt
5f2b6ae Segundo commit - archivo2.txt
3b3463c Primer Commit
```

Otra cosa que se puede hacer con git son poner etiquetas:

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git log --oneline
b437209 (HEAD -> main) Cuarto commit: Ignorar archivo4.txt - .gitignore
d7e0985 Tercer commit - archivo3.txt
5f2b6ae Segundo commit - archivo2.txt
3b3463c Primer Commit

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git tag "primer_gitignore"

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git log --oneline
b437209 (HEAD -> main, tag: primer_gitignore) Cuarto commit: Ignorar archivo4.tx
t - .gitignore
d7e0985 Tercer commit - archivo3.txt
5f2b6ae Segundo commit - archivo2.txt
3b3463c Primer Commit
```

Para ver los tag que hemos puesto podemos poner:

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git tag
primer_gitignore
```

Con este tag, podemos movernos más fácilmente a ese commit en específico.

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git checkout primer_gitignore
Note: switching to 'primer_gitignore'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at b437209 Cuarto commit: Ignorar archivo4.txt - .gitignore
```


También podemos cerrar diferentes ramas independientes unas de otras, facilitando el trabajo en equipo.

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git branch ramal

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git branch
* main
  ramal
```

Nos cambiamos a la rama 1 con el comando: git switch ramal

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git switch ramal
Switched to branch 'ramal'
```

Si creamos un archivo y hacemos un commit, y vemos los logs de los commits, sale como el último commit el que hemos hecho en la rama1.

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (ramal)
$ echo "Primer commit de la ramal" >> archivo5.txt

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (ramal)
$ git add .
warning: in the working copy of 'archivo5.txt', LF will be replaced by CRLF the
next time Git touches it

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (ramal)
$ git commit -m "Primer commit de la ramal"
[ramal b1ce57c] Primer commit de la ramal
1 file changed, 1 insertion(+)
create mode 100644 archivo5.txt

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (ramal)
$ git log --oneline
b1ce57c (HEAD -> ramal) Primer commit de la ramal
b437209 (tag: primer_gitignore, main) Cuarto commit: Ignorar archivo4.txt - .git
ignore
d7e0985 Tercer commit - archivo3.txt
5f2b6ae Segundo commit - archivo2.txt
3b3463c Primer Commit
```

Si hacemos un ls en la rama podemos ver como tenemos 5 archivos, pero si nos vamos a la rama main, y volvemos a hacer ls, vemos que solo tenemos 4.

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (ramal)
$ ls
archivo1.txt  archivo2.txt  archivo3.txt  archivo4.txt  archivo5.txt

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (ramal)
$ git switch main
Switched to branch 'main'

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ ls
archivo1.txt  archivo2.txt  archivo3.txt  archivo4.txt
```

Con el git merge podemos pasar lo que tenemos en la rama, a la rama main.

-Solucionar conflictos: Creamos un archivo con el mismo nombre que el que creamos en la rama.

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git log --oneline
b437209 (HEAD -> main, tag: primer_gitignore) Cuarto commit: Ignorar archivo4.tx
t - .gitignore
d7e0985 Tercer commit - archivo3.txt
5f2b6ae Segundo commit - archivo2.txt
3b3463c Primer Commit

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ echo "Hacemos un commit en la rama main" >> archivo5.txt

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git add .
warning: in the working copy of 'archivo5.txt', LF will be replaced by CRLF the
next time Git touches it

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git commit -m "Hacemos un commit en la rama main en un archivo con el mismo no
mbre que el de la rama"
[main 4aa1040] Hacemos un commit en la rama main en un archivo con el mismo nomb
re que el de la rama
1 file changed, 1 insertion(+)
 create mode 100644 archivo5.txt

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git log --oneline
4aa1040 (HEAD -> main) Hacemos un commit en la rama main en un archivo con el mi
smo nombre que el de la rama
b437209 (tag: primer_gitignore) Cuarto commit: Ignorar archivo4.txt - .gitignore
d7e0985 Tercer commit - archivo3.txt
5f2b6ae Segundo commit - archivo2.txt
3b3463c Primer Commit
```

Ahora nos vamos a la rama:

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git switch ramal
Switched to branch 'ramal'
```

hacemos un merge. Nos sale que hay un conflicto.

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (ramal)
$ git merge main
Auto-merging archivo5.txt
CONFLICT (add/add): Merge conflict in archivo5.txt
Automatic merge failed; fix conflicts and then commit the result.

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (ramal|MERGING)
$ .....
```

Si abrimos el archivo: nano archivo5.txt, nos salen las líneas donde hay conflicto. Aquí podríamos eliminar una de ellas, o dejar las dos.

```
GNU nano 7.2                                archivo5.txt
<<<<<<< HEAD
Primer commit de la rama1
=====
Hacemos un commit en la rama main
>>>>>> main
```

Una vez hecho lo que queramos hacer, lo guardamos y seguimos con el merge.

```
GNU nano 7.2                                archivo5.txt                                Modified
Primer commit de la rama1
Hacemos un commit en la rama main

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (rama1|MERGING)
$ git add .

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (rama1|MERGING)
$ git commit -m "Conflicto resuelto"
[rama1 0558091] Conflicto resuelto

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (rama1)
$ git log --oneline
0558091 (HEAD -> rama1) Conflicto resuelto
4aa1040 (main) Hacemos un commit en la rama main en un archivo con el mismo nomb
re que el de la rama
b1ce57c Primer commit de la rama1
b437209 (tag: primer_gitignore) Cuarto commit: Ignorar archivo4.txt - .gitignore
d7e0985 Tercer commit - archivo3.txt
5f2b6ae Segundo commit - archivo2.txt
3b3463c Primer Commit
```

Git stash : si hemos creado un archivo y esta a medias, por lo que no queremos hacer un commit, podemos guardarlo en un stash:

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (rama1)
$ echo "Archivo a medio terminar" >> archivo6.txt

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (rama1)
$ git stash
Saved working directory and index state WIP on rama1: 0558091 Conflicto resuelto

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (rama1)
$ git stash list
stash@{0}: WIP on rama1: 0558091 Conflicto resuelto

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (rama1)
$ git switch main
Switched to branch 'main'
```

Si volvemos a la rama y hacemos un status vemos que no está el archivo6.txt, y es por que está en el stash. Para traerlo de vuelta tenemos que poner:

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (rama1)
$ git stash pop
On branch rama1
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   archivo6.txt

Dropped refs/stash@{0} (0f4b53f5276a57bd15baca16809787f4c75abf68)
```

Y si intentamos volver a ver las listas de stash, ya no estará la que acabamos de crear.

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (rama1)
$ git stash list
```

También podemos borrar los stash que tengamos con git stash drop:

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (rama1)
$ git stash
Saved working directory and index state WIP on rama1: 0558091 Conflicto resuelto

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (rama1)
$ git stash list
stash@{0}: WIP on rama1: 0558091 Conflicto resuelto

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (rama1)
$ git stash drop
Dropped refs/stash@{0} (7a60b2724df5cab1b641854bc9b6b946342c352d)

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (rama1)
$ git stash list
```

Para unir la rama1 con la rama main tenemos que hacer un merge. Antes de hacerlo podemos hacer un git diff rama1, para ver si hay algo diferente, y si no lo hay o no nos importa la diferencia, podemos hacer un merge.

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git diff rama1
diff --git a/archivo5.txt b/archivo5.txt
index f43d863..eab1d67 100644
--- a/archivo5.txt
+++ b/archivo5.txt
@@ -1,2 +1 @@
-Primer commit de la rama1
+Hacemos un commit en la rama main

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git merge rama1
Updating 4aa1040..0558091
Fast-forward
 archivo5.txt | 1 +
 1 file changed, 1 insertion(+)
```

Como ya las hemos fusionado ya podemos borrar la rama1, y lo hacemos así:

```
judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git branch
* main
  rama1

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git branch -D rama1
Deleted branch rama1 (was 0558091).

judit@PC-Judit MINGW64 /d/DAW/Repositorio/2DAW/practicas_Git (main)
$ git branch
* main
```